vector that represents its guess for the values of each of the possible actions. At first, the model will output a bunch of random values since it is not trained.

We'll run the softmax function over the model's output to generate a probability distribution over the actions. We'll then select an action using the environment's `choose_arm(...)` function, which will return the reward generated for taking that action; it will also update the environment's current state. We'll turn the reward (which is a non-negative integer) into a one-hot vector that we can use as our training data. We'll then run one step of backpropagation with this reward vector, given the state we gave the model. Since we're using a neural network model as our action-value function, we no longer have any sort of action-value array storing "memories;" everything is being encoded in the neural network's weight parameters. The whole `train` function is shown in the following listing.

Listing 2.10. The main training loop

```
def train(env, epochs=5000, learning_rate=1e-2):
    cur_state = torch.Tensor(one_hot(arms,env.get_state()))
1
    optimizer = torch.optim.Adam(model.parameters(),
lr=learning_rate)
    rewards = []
    for i in range(epochs):
        y_pred = model(cur_state)
2
        av_softmax = softmax(y_pred.data.numpy(), tau=2.0)
3
        av_softmax /= av_softmax.sum()
4
        choice = np.random.choice(arms, p=av_softmax)
5
        cur_reward = env.choose_arm(choice)
6
        one_hot_reward = y_pred.data.numpy().copy()
7
        one_hot_reward[choice] = cur_reward
8
        reward = torch.Tensor(one_hot_reward)
        rewards.append(cur_reward)
        loss = loss_fn(y_pred, reward)
        optimizer.zero_grad()
```